Assignment: Fleet Radar (Full-Stack)

Please spend up to 4 hours on this assignment

Objective

Build a small web-based prototype of a Fleet Radar (similar to flight radar, but for remote EVs).

The goal is to demonstrate:

- Full-stack capability (backend + frontend)
- Event-driven thinking
- Real-time state handling
- Pragmatic architectural decisions
- Operator-focused UX thinking (clarity over polish)

Context (User Story)

A fleet operator needs to monitor the vehicle fleet in real time. They need to:

- Understand where cars are located
- See what each car is currently doing
- Identify vehicles that may need charging
- Send a field agent in case of an issue
- Spot areas with low vehicle coverage
- Support a customer trip if something goes wrong

This story is meant to inspire your design thinking. It does not need to be fully implemented, but your solution should reflect operator-oriented decision making.

Functional Requirements

Build a web application that:

- Displays approximately 100 vehicles on a map.
- Each vehicle has live state:
    - Location (lat/lng)
    - Direction / heading
    - Battery percentage (EV)
    - Status: FREE, WITH_CUSTOMER, EN_ROUTE
- Vehicles in EN_ROUTE status display the route they are following.
- The UI updates as vehicle state changes.

Includes simple operator-oriented UX decisions (e.g., legend, selection, filtering, stale indicator, etc.). It's your choice.

Assume that at any given time: ~10 vehicles are simultaneously EN_ROUTE.

Visual polish is not important. Clarity and usability are. We will evaluate the quality of the decisions.

Event-Driven Architecture Requirement

Design your backend assuming:

- Vehicle telemetry (location, heading, battery, status) arrives as events over Kafka.
- Route assignments/updates also arrive as events.

You do not need to run Kafka itself. A simulated event source is sufficient.

However:

- Structure your backend as if Kafka events are the source of truth.
- Model your data flow accordingly.

Technical Constraints

- Technology choices are fully open.
- The solution must run locally.
- In-memory storage is acceptable.
- No authentication required.

Deliverables

Provide:

- A repository or zip file containing:
  - Backend code
  - Frontend code
- Instructions to run locally
- A short README including:
  - How to run the project
  - A brief description of your architecture and data flow (for ~100 vehicles)
  - Key tradeoffs made within the timebox

Be prepared to discuss how your design would scale to ~1000 vehicles (no implementation required).